Отчёт по лабораторной работе № 11 (Lab11)
Бехтеев Дмитрий Николаевич

Тема: Двумерные массивы

Цель работы:

1. Анализ строк и столбцов матрицы по заданным условиям (наличие или отсутствие нулей)
2. Поиск элементов с определенной частотностью (встречающихся более одного раза)
3. Вычисление характеристик строк на основе их элементов
4. Сортировка данных матрицы по вычисленным характеристикам
5. Определение серий одинаковых элементов внутри строк

СДЕЛАТЬ ПОЛОСКУ РАЗДЕЛИТЕЛЬНУЮ

Задача 1: Дана целочисленная прямоугольная матрица:

```
        static void Variant1()
{
    {
        Console.Write("Введите количество строк: ");
        int rows = int.Parse(Console.ReadLine());

        Console.Write("Введите количество столбцов: ");
        int cols = int.Parse(Console.ReadLine());

        int[,] matrix = new int[rows, cols];

        Console.WriteLine("Введите элементы матрицы:");
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
            {
                Console.Write($"matrix[{i},{j}] = ");
                matrix[i, j] = int.Parse(Console.ReadLine());
            }
        }

        int rowsWithoutZero = 0;
        for (int i = 0; i < rows; i++)
        {
            bool hasZero = false;
            for (int j = 0; j < cols; j++)
            {
                if (matrix[i, j] == 0)
                {
                    hasZero = true;
                    break;
                }
            }
            if (!hasZero)
                rowsWithoutZero++;
        }

        Console.WriteLine($"1. Количество строк без нулевых элементов: {rowsWithoutZero}");

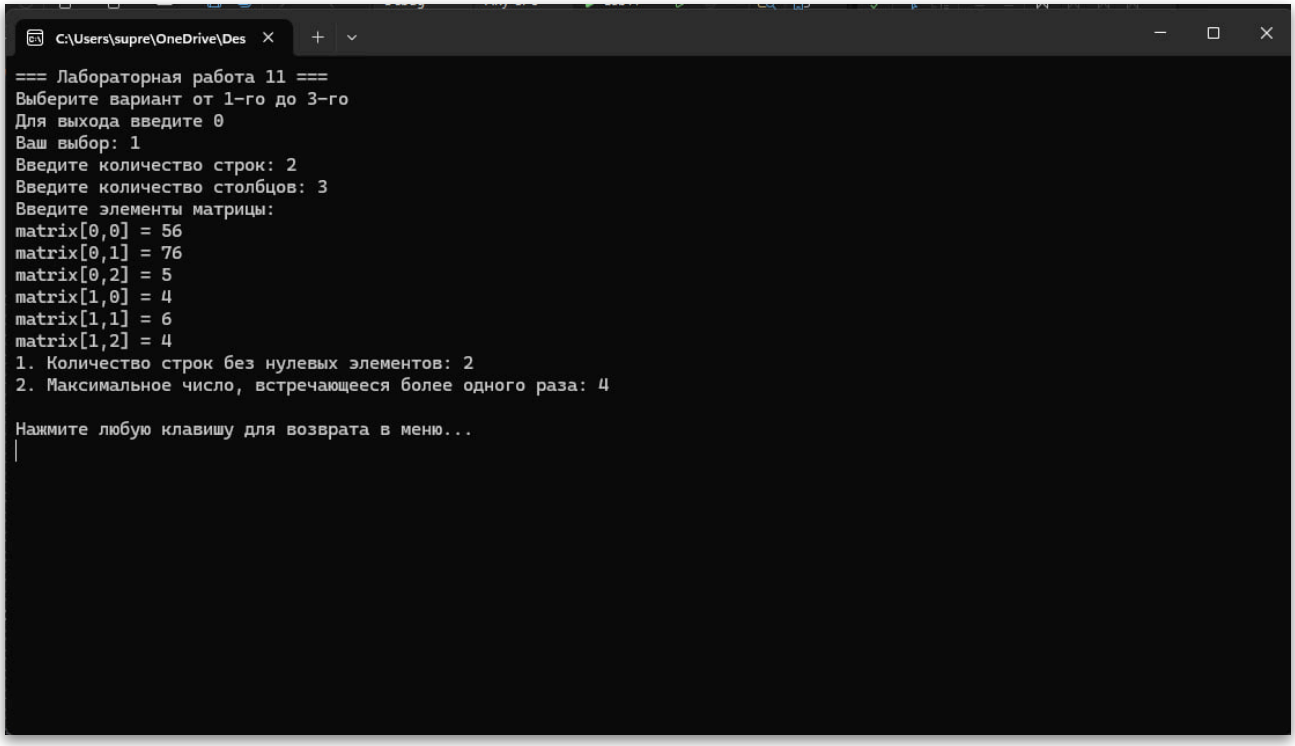
        Dictionary<int, int> frequency = new Dictionary<int, int>();

        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
            {
                int num = matrix[i, j];
                if (frequency.ContainsKey(num))
                    frequency[num]++;
                else
```

```
                frequency[num] = 1;
            }
        }

        int maxRepeated = int.MinValue;
        foreach (var pair in frequency)
        {
            if (pair.Value > 1 && pair.Key > maxRepeated)
            {
                maxRepeated = pair.Key;
            }
        }

        if (maxRepeated != int.MinValue)
            Console.WriteLine($"2. Максимальное число, встречающееся более одного раза: {maxRepeated}");
        else
            Console.WriteLine("2. Нет чисел, встречающихся более одного раза");
    }
}
```



Алгоритм:

1. Ввод матрицы: запрашиваем размеры M×N и заполняем значениями
2. Строки без нулей: для каждой строки проверяем наличие нулей. Если нулей нет → увеличиваем счетчик `rowsWithoutZero`
3. Частотный анализ: создаем словарь для подсчета повторений
   - Проходим по всем элементам матрицы
   - Для каждого числа увеличиваем счетчик в словаре
4. Поиск максимума: в словаре ищем числа с частотой >1
   - Выбираем максимальное из таких чисел
5. Вывод результатов: показываем количество строк без нулей и найденный максимум

Задача 2: Дана целочисленная прямоугольная матрица:

```
    static void Variant2()
{
    Console.Write("Введите количество строк: ");
    int rows = int.Parse(Console.ReadLine());

    Console.Write("Введите количество столбцов: ");
    int cols = int.Parse(Console.ReadLine());

    int[,] matrix = new int[rows, cols];

    Console.WriteLine("Введите элементы матрицы:");
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            Console.Write($"matrix[{i},{j}] = ");
            matrix[i, j] = int.Parse(Console.ReadLine());
        }
    }

    int colsWithoutZero = 0;
    for (int j = 0; j < cols; j++)
    {
        bool hasZero = false;
        for (int i = 0; i < rows; i++)
        {
            if (matrix[i, j] == 0)
            {
                hasZero = true;
                break;
            }
        }
        if (!hasZero)
            colsWithoutZero++;
    }

    Console.WriteLine($"1. Количество столбцов без нулевых элементов: {colsWithoutZero}");

    int[] characteristics = new int[rows];

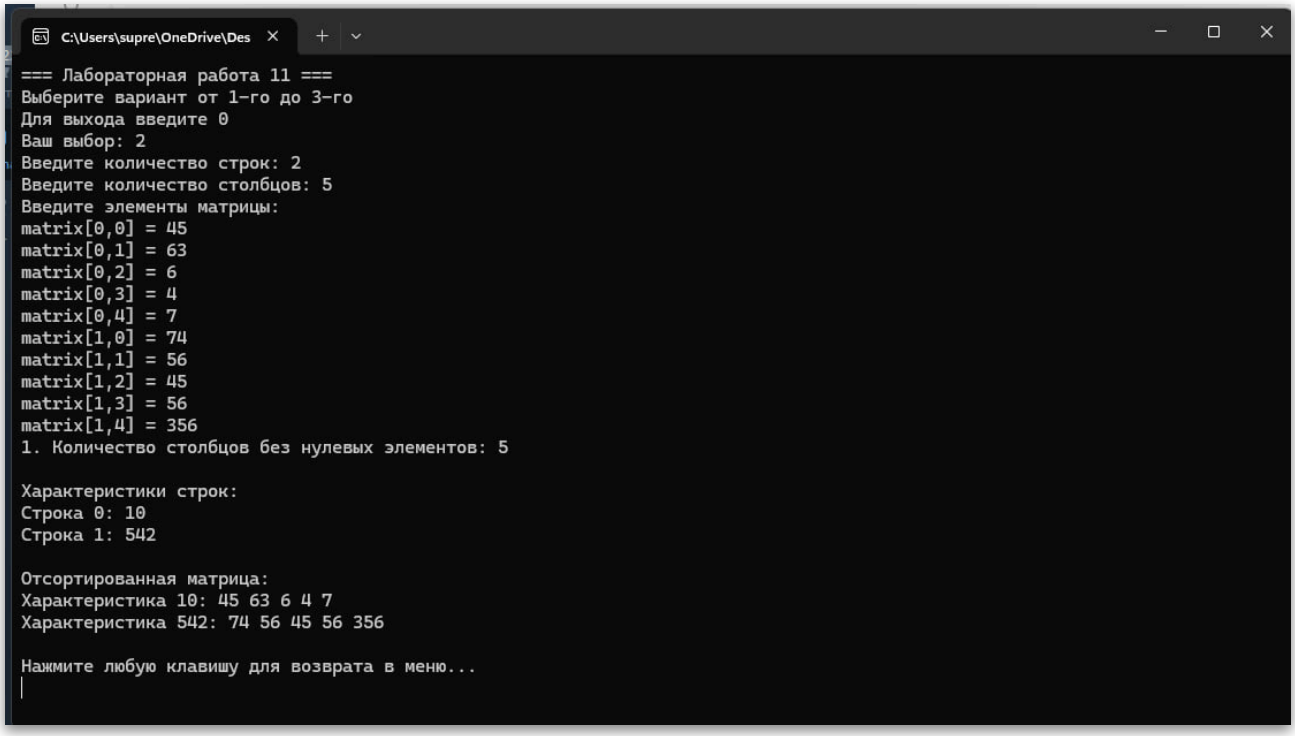
    for (int i = 0; i < rows; i++)
    {
        int sum = 0;
        for (int j = 0; j < cols; j++)
        {
            if (matrix[i, j] > 0 && matrix[i, j] % 2 == 0)
            {
                sum += matrix[i, j];
            }
        }
        characteristics[i] = sum;
    }

    int[] rowIndices = new int[rows];
    for (int i = 0; i < rows; i++)
        rowIndices[i] = i;

    for (int i = 0; i < rows - 1; i++)
    {
        for (int j = 0; j < rows - i - 1; j++)
        {
            if (characteristics[rowIndices[j]] > characteristics[rowIndices[j + 1]])
            {
                int temp = rowIndices[j];
                rowIndices[j] = rowIndices[j + 1];
                rowIndices[j + 1] = temp;
            }
        }
    }
```

```
    Console.WriteLine("\nХарактеристики строк:");
    for (int i = 0; i < rows; i++)
    {
        Console.WriteLine($"Строка {i}: {characteristics[i]}");
    }

    Console.WriteLine("\nОтсортированная матрица:");
    for (int i = 0; i < rows; i++)
    {
        int rowIndex = rowIndices[i];
        Console.Write($"Характеристика {characteristics[rowIndex]}: ");
        for (int j = 0; j < cols; j++)
        {
            Console.Write(matrix[rowIndex, j] + " ");
        }
        Console.WriteLine();
    }
}
```



Алгоритм:

1. Столбцы без нулей: для каждого столбца проверяем все элементы
2. Вычисление характеристик строк: для каждой строки
3. Сортировка по характеристикам: используем сортировку
4. Вывод результатов:
   - Количество столбцов без нулей
   - Характеристики всех строк
   - Отсортированная матрица (вывод строк в порядке возрастания характеристик)

Задача 3: Дана целочисленная прямоугольная матрица:

```
        static void Variant3()
{
    {
        Console.Write("Введите количество строк: ");
        int rows = int.Parse(Console.ReadLine());

        Console.Write("Введите количество столбцов: ");
        int cols = int.Parse(Console.ReadLine());

        int[,] matrix = new int[rows, cols];

        Console.WriteLine("Введите элементы матрицы:");
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < cols; j++)
            {
                Console.Write($"matrix[{i},{j}] = ");
                matrix[i, j] = int.Parse(Console.ReadLine());
            }
        }

        int colsWithZero = 0;
        for (int j = 0; j < cols; j++)
        {
            bool hasZero = false;
            for (int i = 0; i < rows; i++)
            {
                if (matrix[i, j] == 0)
                {
                    hasZero = true;
                    break;
                }
            }
            if (hasZero)
                colsWithZero++;
        }

        Console.WriteLine($"1. Количество столбцов с нулевыми элементами: {colsWithZero}");

        int maxSeriesRow = 0;
        int maxSeriesLength = 1;

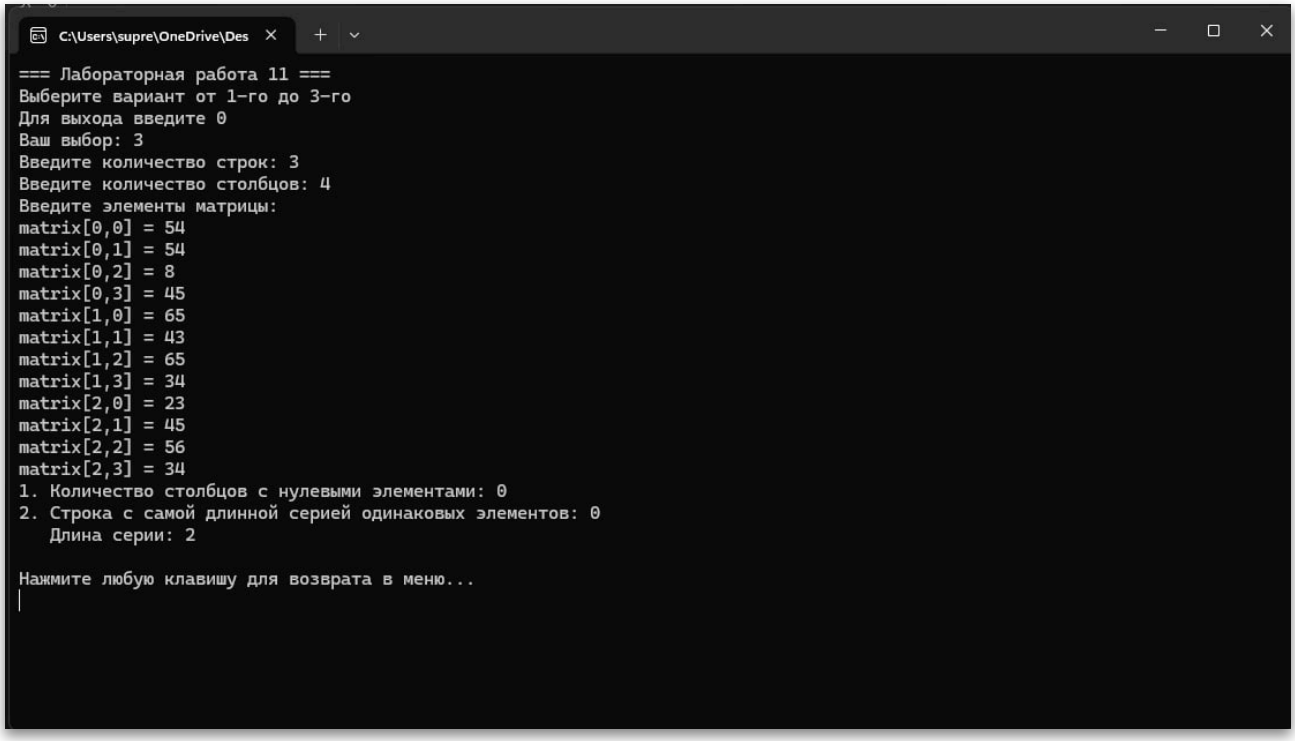
        for (int i = 0; i < rows; i++)
        {
            int currentSeriesLength = 1;
            int currentMaxSeries = 1;

            for (int j = 1; j < cols; j++)
            {
                if (matrix[i, j] == matrix[i, j - 1])
                {
                    currentSeriesLength++;
                    if (currentSeriesLength > currentMaxSeries)
                        currentMaxSeries = currentSeriesLength;
                }
                else
                {
                    currentSeriesLength = 1;
                }
            }

            if (currentMaxSeries > maxSeriesLength)
            {
                maxSeriesLength = currentMaxSeries;
                maxSeriesRow = i;
            }
        }

        Console.WriteLine($"2. Строка с самой длинной серией одинаковых элементов: {maxSeriesRow}");
        Console.WriteLine($"   Длина серии: {maxSeriesLength}");
```

```
    }
}
```



Алгоритм:

1. Столбцы с нулями: для каждого столбца проверяем все элементы сверху вниз
2. Поиск самой длинной серии: для каждой строки:
3. Определение строки-победителя: сравниваем `currentMaxSeries` всех строк

4. Вывод результатов:
   - Количество столбцов, содержащих нули
   - Номер строки с самой длинной серией и длина этой серии